GenAI Conversation Export Logs

COMP3011 – Web Services and Web Data

Metadata

Field	Value
Module	COMP3011 – Web Services and Web Data
Student	Nathaniel Sebastian (sc232ns)
Date	17th March 2026
Tools Used	Google Gemini (Antigravity), Claude (Anthropic), ChatGPT (OpenAI)

Summary Table

Tool	Purpose	Sessions
Google Gemini	Architecture, coding, debugging, test generation, security hardening	14
Claude	Documentation polish, refactoring review, portability analysis	4
ChatGPT	Early brainstorming, schema design	2

Test Count: 84 tests passing (verified via `pytest -q`)

---

1. High-Level Design Decisions

Decision 1: Middleware Ordering for Security Headers

- **Problem:** Headers not applied to all response types (429, 500, 404)
- **Options:**
  - (a) Per-route decorator
  - (b) Exception handler only
  - (c) Middleware with `add_headers` helper
- **Decision:** Option (c). Created `add_headers(resp)` function called on all paths.
- **Verification:** `tests/test_headers_security.py` confirms headers on 200, 404.

Decision 2: ETag Implementation Approach

- **Problem:** Need conditional GET (304 Not Modified)
- **Options:**
  - (a) Database `updated_at` column
  - (b) Response body SHA256 hash
- **Decision:** Option (b). Middleware computes SHA256, checks `If-None-Match`, returns 304. Based on RFC 7232.

Decision 3: Analytics Portability (SQLite vs PostgreSQL)

- **Problem:** `strftime` is SQLite-only, but prod uses PostgreSQL
- **AI suggested:** Just use `strftime` everywhere — **AI WAS WRONG**
- **Rejected AI suggestion:** `strftime` does not exist in PostgreSQL
- **Fix:** Created `_month_expr()` helper that detects dialect and uses `strftime` for SQLite, `to_char(date_trunc())` for PostgreSQL
- **Note:** This is an example of **REJECTING** a weaker AI suggestion

---

2. Creative AI Use — Event Ownership Design

- Used AI to explore ownership models:
  - (a) Separate ACL table
  - (b) `created_by` FK
  - (c) Team-based
- AI recommended option (b) for coursework scope
- Implemented `created_by_user_id` with ownership checks on PATCH/DELETE
- This is an example of using AI **creatively** for design exploration, not just debugging

---

3. Session Logs (Brief Summaries)

Session	Tool	Topic
Architecture planning	Gemini	Layered architecture adopted
RSVP storage	Gemini	Separate table for data integrity
Rate limiting strategy	Gemini	In-memory for coursework
ETag implementation	Gemini	RFC 7232 compliance
Provenance design	Claude	Import tracking with SHA256, parser versioning
Error sanitisation	Gemini	Middleware catch-all pattern
Test coverage improvement	Claude	Branch coverage analysis and targeted test generation

---

4. Failures & Corrections

AI Failure	Impact	My Fix
Missing `requests` in requirements.txt	ModuleNotFoundError	Added manually
Placeholder test with `pass`	False coverage	Rewrote with assertions
Deprecated `Query(regex=...)`	Warning	Changed to `pattern=...`
Headers not on 429 responses	Missing security	Fixed middleware flow

AI Failure	Impact	My Fix
`strftime` claimed to work on PostgreSQL	Would crash in prod	Created dialect-aware `_month_expr()`
Suggested overly broad `# type: ignore` comments	Masked real issues	Used specific typing fixes instead

---

5. Validation Steps

1. Clean install test
2. Full test suite: `pytest -q` → 84 passed
3. Quality gates: `ruff`, `mypy`, `bandit` all pass
4. Manual curl testing of all endpoints
5. ETag verification
6. Ownership/authz verification

---

6. Conclusion

AI accelerated development approximately 3× but required significant manual verification. Critical bugs were caught via clean-install testing and explicit test cases. The most important lesson was that AI-suggested portability claims must be verified against actual database dialect documentation.